OBJECT-ORIENTED PROGRAMMING

Winter semester of academic year 2019-2020

Third Assignment

This assignment [1] aims to give you practice in inheritance, virtual functions and abstract classes. It is inspired by the prebuilt class hierarchy of object-oriented programming languages such as Java and C#. In these languages, there is a predefined class hierarchy and every new class that will be defined by the user must be integrated into this hierarchy.

Implement in C++ the following class hierarchy. There is a root class of the hierarchy, the class "Object". Every other class is a subclass of it. Every object of this class has an identity (`id`) that identifies it uniquely. If an object is a copy of another, it will have the same identity as it. Comparison of objects is done either by comparing them to see if they are the same by value (`equal`) or by checking if they are actually the same entities in memory (`identical`). Every object provides, as well, the ability to create a copy of itself (`clone`). Finally, every object provides a description of itself in string form by converting all of its data members to strings and concatenating them (`toString`). No objects of this class are defined.

The class "String" is a subclass of the class "Object" and models a sequence of characters (*Note*: you are free to follow any approach for the internal representation). For every object of this class we can find out its length (`length`), empty it (`clear`), concatenate it with some other one (`concat`), get the character located at a specific position in the string (`at`) and change the character at this position (`updateAt`). Also, we can print the contents of a string to the standard output (`print`).

The above classes that you are asked to implement can be considered to be from the repertoire of the classes of the language. In this assignment, you are asked, beyond these, to also implement the following classes, which can be considered to appear in a broader application that you are implementing. Suppose, that we have the concept of the plane, the concept of the parts of the plane as well as a hierarchy of concepts of employees who work on parts of the plane.

A "Plane" consists of various components/parts that compose it and model spaces of the plane. A "Plane Component" (PlaneComponent) is a subclass of the class "Object". It has a description and provides the ability to confirm the readiness for use of the space it models (`ready_check`). Finally, it is given an employee to work on it (`process`). No objects of this class are defined.

A "Passenger Compartment" (PassengerCompartment) is a "Plane Component" that possibly also has some inner "Passenger Compartment". During the confirmation of its readiness for use (`ready_check`), the description of the space is displayed, the message `"Passenger Compartment OK!"` is printed and this procedure continues for its inner space, if it exists. An employee works on this space (`process`), by being assigned this space to work on. Finally, the procedures that must be implemented in order for objects of it to be defined are implemented, in the obvious way.

Correspondingly to the class "Passenger Compartment" a class "Restricted Access Compartment" (PrivateCompartment) is defined. That is, it is a "Plane Component" but without any inner space. Simply, it exhibits all of the above behavior, though, in an analogous way. Special cases of this class are the classes "Equipment Compartment" (EquipmentCompartment) and "Cargo Bay" (CargoBay) in which simply the messages differ. Also, the "Cargo Bay" contains an "Equipment Compartment" as well.

The "Plane" is an "Object". Moreover, it has a description, a title and a passenger capacity. Furthermore, it has a "Cargo Bay" and three "Equipment Compartments". Also, it has some number of "Passenger Compartments" as well. The exact number is determined as a function of

[1] I thank Giorgos Kastrinis for his contribution to the specifications and to the wording of this assignment

the capacity of the plane but is chosen randomly. For all of these the ability to retrieve their value is given. On the "Plane" a confirmation of its readiness for use is performed (`ready_check`). The "Plane" is given an employee to work on it (`process`).

In the application there is also the class "Employee". Every "Employee" is characterized by their name. An "Employee" is assigned a plane space to work on (`workOn`). An "Employee" reports on their tasks (`report`). No objects of this class are defined.

There are three categories of employees. The "Security Employee" (`SecurityEmployee`) works (`workOn`) on every component of the plane performing checks (printing an appropriate message). The "Maintenance Employee" (`MaintenanceEmployee`) performs maintenance tasks on cargo bays and on equipment compartments. Finally, the "Cleaning Employee" (`CleaningEmployee`) works on passenger compartments and on cargo bays. Every kind of Employee, reports on their tasks (`report`), e.g. by printing a message with the kind of task they perform and the message `"OK"`.

Beyond the above classes, implement a function `clone_encrypt_and_print` that takes a reference to an "Object" and does the following. It creates a copy of the Object, checks them against each other, converts them to strings, changes the string of the new one by changing characters at positions that are chosen randomly and, finally, prints the final strings. Then, it concatenates them, prints the length of the concatenation, prints the middle character(s), empties it and prints its length again.

Implement a function `main`, which creates an object of type "Plane" as well as one object of each kind of "Employee". Then, each employee is given to the plane to work on it and afterwards each employee reports on their tasks. Finally, a confirmation of its readiness for use is performed and the message `"Plane ready to take off!"` is printed.

Additionally, in `main`, the function `clone_encrypt_and_print` is called for this plane.

Note 1: For all data members provide member functions that return their value.

Note 2: In all classes there should be constructors and destructors that, beyond the necessary functionality if any, print on the screen the message `<Object> just created!` and `<Object> to be destroyed!` for constructors and destructors, respectively, where `<Object>` will be the name of the class each time.

Note 3: Each class of the above, should offer externally only the functionality that was described and nothing more.